FPS Data Provider

FPS (frames per second) data transfer between iCUE and HTML widgets: current FPS value, availability status, and current foreground process name.

Overview

- **Module name**: `widgetbuilder.fpsdataprovider`
- **Plugin name**: `Fps`
- **Version**: `1.0`

Manifest entry:

```
"required_plugins": [
  "widgetbuilder.fpsdataprovider:Fps:1.0"
]
```

All data retrieval methods are asynchronous using `requestId` for correlation.

Async Request Pattern

1. Call a method with unique `requestId`.
2. Listen for `asyncResponse(requestId, value)` signal.
3. Match response `requestId` to request.

```
let nextRequestId = 0;
const requestId = nextRequestId++;
window.plugins.Fpsdataprovider.asyncResponse.connect((id, value) => {
    if (id === requestId) {
        console.log("FPS value:", value);
    }
});
window.plugins.Fpsdataprovider.getCurrentFps(requestId);
```

Properties

Property	Type	Description
`currentFps`	`int`	Current frames per second
`fpsAvailable`	`bool`	Whether FPS data is currently available
`currentProcess`	`string`	Current foreground process name

Methods

`getCurrentFps(requestId)`

Gets current FPS value.

Parameter	Type	Description
`requestId`	`int`	Request ID

Response: `int` - Current frames per second

`getFpsAvailable(requestId)`

Checks whether FPS data is currently available.

Parameter	Type	Description
`requestId`	`int`	Request ID

Response: `bool` - Whether FPS data is available

`getCurrentProcess(requestId)`

Gets the name of the current foreground process.

Parameter	Type	Description
`requestId`	`int`	Request ID

Response: `string` - Current foreground process name (file description from the executable).

Example:

```
let nextRequestId = 0;
const requestId = nextRequestId++;
window.plugins.Fpsdataprovider.asyncResponse.connect((id, value) => {
    if (id === requestId) {
        console.log("Current process:", value);
    }
});
window.plugins.Fpsdataprovider.getCurrentProcess(requestId);
```

Signals

`asyncResponse(requestId, value)`

Emitted when an async method completes.

Parameter	Type	Description
`requestId`	`int`	Original request ID
`value`	`var`	Response value

`fpsUpdated(fps)`

Emitted when the FPS value changes.

Parameter	Type	Description
`fps`	`int`	New FPS value

`fpsAvailabilityChanged(available)`

Emitted when FPS data availability changes.

Parameter	Type	Description
`available`	`bool`	Whether FPS data is available

`processChanged(process)`

Emitted when the foreground process name changes.

Parameter	Type	Description
`process`	`string`	New foreground process name

SimpleFpsApiWrapper

Promise-based wrapper for the FPS plugin. Converts the callback-based Qt async API into Promises.

Important: Use Local Wrapper Files

Copy `common/plugins/` from the documentation bundle into your widget folder and include wrappers via local `<script src>` paths.

Do not reference iCUE installation paths (for example `../common/plugins/...`) in third-party widgets.

Initialization

```
const api = new SimpleFpsApiWrapper(window.plugins.Fpsdataprovider);
```

Parameter	Type	Default	Description
`plugin`	`object`	-	Plugin instance (`window.plugins.Fpsdataprovider`)
`timeoutMs`	`number`	`5000`	Request timeout (ms)

Methods

All methods return a `Promise`.

Method	Returns	Description
`getCurrentFps()`	`Promise<int>`	Current FPS value
`getFpsAvailable()`	`Promise<boolean>`	Whether FPS data is available
`getCurrentProcess()`	`Promise<string>`	Current foreground process name

Required local files

```
<script src="common/plugins/IcueWidgetApiWrapper.js"></script>
<script src="common/plugins/SimpleFpsApiWrapper.js"></script>
```

Example

`manifest.json`

```
"required_plugins": [
  "widgetbuilder.fpsdataprovider:Fps:1.0"
]
```

`index.html`

```
// After including IcueWidgetApiWrapper and SimpleFpsApiWrapper in <head>:
const fpsApi = new SimpleFpsApiWrapper(window.plugins.Fpsdataprovider);

async function displayFpsData() {
    const available = await fpsApi.getFpsAvailable();
    if (available) {
        const [fps, process] = await Promise.all([
            fpsApi.getCurrentFps(),
            fpsApi.getCurrentProcess()
        ]);
        console.log(`${process}: ${fps} FPS`);
    }
}
displayFpsData();
```